



Programming In C

**Edited By:
Balraj Kumar**

Title: PROGRAMMING IN C

Author's Name: Dr. Prikshat Kumar Angra

Published By : Lovely Professional University

Publisher Address: Lovely Professional University, Jalandhar Delhi GT road, Phagwara - 144411

Printer Detail: Lovely Professional University

Edition Detail: (I)

ISBN: 978-93-94068-23-0



Copyrights@ Lovely Professional University

CONTENT

Unit 1:	Introduction	1
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 2:	Data Types	18
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 3:	Input/Output in C	30
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 4:	Operations	46
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 5:	Decision making statements	58
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 6:	Loop Statements	84
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 7:	Functions	105
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 8:	Storage Classes	123
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 9:	Arrays	137
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 10:	Pointers	152
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 11:	Strings	168
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 12:	Dynamic Memory Management	184
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 13:	Structure and Union	196
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	
Unit 14:	Structures	219
	<i>Prikshat Kumar Angra, Lovely Professional University</i>	

Unit 01: Introduction

CONTENTS

Objectives

Introduction

- 1.1 Programming Language
- 1.2 Machine Level Language
- 1.3 Advantages and Limitations of Machine Language
- 1.4 Assembly Language
- 1.5 Assembly Program Execution
- 1.6 High Level Languages
- 1.7 Introduction to C Programming
- 1.8 Origin and Development of C Language
- 1.9 Applications of C Language
- 1.10 Evolution of C
- 1.11 Compiler and Interpreter
- 1.12 Program Development in C
- 1.13 Writing a C Program
- 1.14 C Character set
- 1.15 Identifiers and keywords

Summary

Keywords

Self-Assessment

Answer for Self Assessment

Further Readings

Objectives

After studying this unit, you will be able to:

- Understand Programming language
- Discuss C Programming
- Different stages in program development using code blocks IDE
- Programming methodologies

Introduction

Computer is an electronic device which works on the instructions provided by the user. As the computer does not understand natural language, it is required to provide the instructions in some computer understandable language. Such a computer understandable language is known as Programming language.

A computer programming language consists of a set of symbols and characters, words, and grammar rules that permit people to construct instructions in the format that can be interpreted by the computer system. Computer Programming is the art of making a computer do what you want it

to do. Computer programming is a field that has to do with the analytical creation of source code that can be used to configure computer systems. Computer programmers may choose to function in a broad range of programming functions, or specialize in some aspect of development, support, or maintenance of computers for the home or workplace. Programmers provide the basis for the creation and ongoing function of the systems that many people rely upon for all sorts of information exchange, both business related and for entertainment purposes.

1.1 Programming Language

Different programming languages support different styles of programming. The choice of language used is subject to many considerations, such as company policy, suitability to task, availability of third-party packages, or individual preference. Ideally, the programming language best suited for the task at hand will be selected. Trade-offs from this ideal involve finding enough programmers who know the language to build a team, the availability of compilers for that language, and the efficiency with which programs written in a given language execute.

The basic instructions of programming language are:

1. Input: Get data from the keyboard, a file, or some other device.
2. Output: Display data on the screen or send data to a file or other device.
3. Math: Perform basic mathematical operations like addition and multiplication.
4. Conditional execution: Check for certain conditions and execute the appropriate sequence of statements.
5. Repetition: Perform some action repeatedly, usually with some variation.

1.2 Machine Level Language

Computer language, also known as machine code, is a low-level programming language made up of binary digits (ones and zeros). Before a computer can run code written in high-level languages like Swift and C++, the code must be converted into machine language.

Since computers are digital devices, they only recognize binary data. Every program, video, image, and character of text is represented in binary. This binary data, or machine code, is processed as input by the CPU. The resulting output is sent to the operating system or an application, which displays the data visually. For example, the ASCII value for the letter "A" is 01000001 in machine code, but this data is displayed as "A" on the screen. An image may have thousands or even millions of binary values that determine the color of each pixel.

While computers can be programmed to understand a variety of computer languages, there is only one language that the computer understands without the use of a translation program; this language is known as the computer's machine language or machine code. Machine code is the fundamental language of a computer and is normally written as strings of binary 1s and 0s. The circuitry of a computer is wired in such a way that it immediately recognizes the machine language and converts it into the electrical signals needed to run the computer. An instruction prepared in any language has a two part. The first part is command or operation, and it tells the computer what function to perform. Every computer has an operation code or op-code for each of its functions. The second part of the instruction is the operand, and it tells the computer where to find or store the data or other instructions that are to be maintained. Thus, each instruction tells the control unit of the CPU what to do and the length and location of the data field are involved in the operation. Typical operations involve reading, adding, subtracting, writing and so on.



Discuss Example of Machine Level Language

Instruction Format

We already know that all computers use binary digits (0s and 1s) for performing operations. Hence, most computers machine language consists of strings of binary numbers and is the only one the CPU directly understands. When stored inside the computer, the symbols which make up the machine language program are made up of 1s and 0s.



A typical program instruction to print out a number on the printer might be.

```
101100111111010011101100110000111001
```

The program to add two numbers in memory and print the result look something like the following:

```
001000000000001100111001
001111000000111111000111
100111100011101100110101
101100010101010101110000
000000000000000000000000
```

This is obviously not a very easy language to learn, partly because it is difficult to read and understand and partly because it is written in a number system with which we are not familiar. But it will be surprising to note that some of the first programmers, who worked with the first few computers, actually wrote their programs in binary form as above. Since human programmers are more familiar with the decimal number system, most of them preferred to write the computer instructions in decimal, and leave the input device to convert these to binary. In fact, without too much effort, a computer can be wired so that instead of using long numbers. With this change, the preceding program appears as follows:

```
10001471
14002041
30003456
50773456
00000000
```

The set of instruction codes, whether in binary or decimal, which can be directly understood by the CPU of a computer without the help of a translating program, is called a machine code or machine language. Thus, a machine language program need not necessarily be coded as strings of binary digits (1s and 0s). It can also be written using decimal digits if the circuitry of the computer being used permits this.

1.3 Advantages and Limitations of Machine Language

Programs written in machine language can be executed very fast by the computer. This is mainly because machine instructions are directly understood by the CPU writing a program in machine language has several disadvantages which are discussed below.

1. Machine dependent

Because the internal design of every type of computer is different from every other type of computer and needs different electrical signals to operate, the machine language also is different from computer to computer. It is determined by the actual design or construction of the CPU, the control unit, and the size as well as the word length of the memory unit. Hence, suppose after becoming proficient in the machine code of a particular computer, a company decides to change to another computer, the programmer may be required to learn a new machine language and would have to rewrite all the existing programs.

2. Difficult to program

Although easily used by the computer, machine language is difficult to program, it is necessary for the programmer either to memorize the dozens of code numbers for the commands in the machine's instruction set or to constantly refer to keep track of the storage location of data and instructions. Moreover, a machine language programmer must be an expert who knows about the hardware structure of the computer.

3. Error code

For writing programs in machine language, since a programmer has to remember the opcodes and he must also keep track of the storage location of data and instructions, it becomes very difficult for him to concentrate fully on the logic of the problem. This frequently results in program errors. Hence, it is easy to make errors while using machine code.

4. Difficult to modify

It is difficult to correct or modify machine language programs. Checking machine instructions to locate errors is about as tedious as writing them initially. Similarly, modifying a machine language program at a later date is so difficult that many programmers would prefer to code the new logic afresh instead of incorporating the necessary modifications in the old program.

1.4 Assembly Language

Assembly languages are also known as second generation languages. These languages substitute alphabetic symbols for the binary codes of machine language. In assembly language, symbols are used in place of absolute addresses to represent memory locations. Mnemonics are used for operation code, i.e., single letters or short abbreviations that help the programmers to understand what the code represents.



MOV AX, DX.



Here mnemonic MOV represents 'transfer' operation and AX, DX are used to represent the registers. One of the first steps in improving the program preparation process was to substitute letter symbols mnemonics for the numeric operation codes of machine language. A mnemonic is any kind of mental trick we use to help us remember. Mnemonics come in various shapes and sizes, all of them useful in their own way.

All computers have the power of handling letters as well as numbers. Hence, a computer can be taught to recognize certain combination of letter or numbers. It can be taught to substitute the number 14 every time it sees the symbol ADD, substitute the number 15 every time it sees the symbol SUB, and so forth. In this way, the computer can be trained to translate a program written with symbols instead of numbers into the computer's own machine language. Then we can write program for the computer using symbols instead of numbers, and have the computer do its own translating. This makes it easier for the programmer, because he can use letters, symbols, and mnemonics instead of numbers for writing his programs.



The preceding program that was written in machine language for adding two numbers and printing out the result could be written in the following way:

```
CLA    A
ADD    B
STA    C
TYP    C
```

Which would mean "take A, add B, store the result in C, type C, and halt." The computer by means of a translating program, would translate each line of this program into the corresponding machine language program.

Advantages of Assembly Language

1. Assembly language is easier to use than machine language.
2. An assembler is useful for detecting programming errors.
3. Programmers do not have to know the absolute addresses of data items.
4. Assembly languages encourage modular programming.

Disadvantages of Assembly Language

1. Assembly language programs are not directly executable.
2. Assembly languages are machine dependent and, therefore, not portable from one machine to another.
3. Programming in assembly language requires a higher level of programming skill.

1.5 Assembly Program Execution

An assembly program is written according to a strict set of rules. An editor or word processor is used for keying an assembly program into the computer as a file, and then the assembler is used to translate the program into machine code.

There are two ways of converting an assembly language program into machine language:

1. Manual assembly
2. By using an assembler.

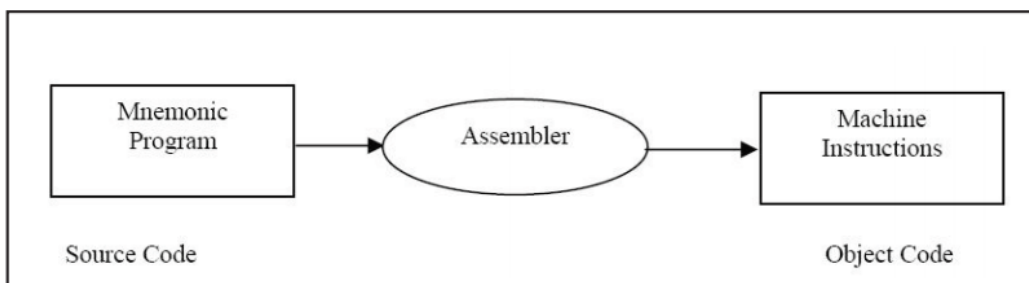
Manual Assembly

It was an old method that required the programmer to translate each opcode into its numerical machine language representation by looking up a table of the microprocessor instructions set, which contains both assembly and machine language instructions. Manual assembly is acceptable for short programs but becomes very inconvenient for large programs. The Intel SDK-85 and most of the earlier university kits were programmed using manual assembly.

Using an Assembler

The symbolic instructions that you code in assembly language is known as - Source program.

An assembler program translates the source program into machine code, which is known as object program.



The steps required to assemble, link and execute a program are:

1. The assembly step involves translating the source code into object code and generating an intermediate .OBJ (object file) or module. The assembler also creates a header immediately in front of the generated .OBJ module; part of the header contains information about incomplete addresses. The .OBJ module is not quite in executable form.
2. The link step involves converting the .OBJ module to an .EXE machine code module. The linker's tasks include completing any address left open by the assembler and combining separately assembled programs into one executable module.



The linker

- (a) Combines assembled module into one executable program
- (b) Generates an .EXE module and initializes with special instructions to facilitate its

3. The last step is to load the program for execution. Because the loader knows where the program is going to load in memory, it is now able to resolve any remaining address still left incomplete in the header. The loader drops the header and creates a program segment prefix (PSP) immediately before the program is loaded in memory.

Tools Required for Assembly Language Programming

The tools of the assembly process described may vary in details.

The editor is a program that allows the user to enter, modify, and store a group of instructions or text under a file name. The editor programs can be classified in two groups.

1. Line editors
2. Full screen editors

Line editors, such as EDIT in MS DOS, work with the manage one line at a time. Full screen editors, such as Notepad, WordPad etc. manage the full screen or a paragraph at a time. To write text, the user must call the editor under the control of the operating system. As soon as the editor program is transferred from the disk to the system memory, the program control is transferred from the operating system to the editor program. The editor has its own command and the user can enter and modify text by using those commands. Some editor programs such as WordPerfect are very easy to use. At the completion of writing a program, the exit command of the editor program will save the program on the disk under the file name and will transfer the control to the operating system. If the source file is intended to be a program in the 8086 assembly language the user should follow the syntax of the assembly language and the rules of the assembler.

Linker

For modularity of your programs, it is better to break your program into several sub routines. It is even better to put the common routine, like reading a hexadecimal number, writing hexadecimal number, etc., which could be used by a lot of your other programs into a separate file. These files are assembled separately. After each file has been successfully assembled, they can be linked together to form a large file, which constitutes your complete program. The file containing the common routines can be linked to your other program also. The program that links your program is called the linker.

Loader

Loader is a program which assigns absolute addresses to the program. These addresses are generated by adding the address from where the program is loaded into the memory to all the offsets. Loader comes into action when you want to execute your program. This program is brought from the secondary memory like disk. The file name extension for loading is .exe or .com, which after loading can be executed by the CPU.

Differences between Machine-Level language and Assembly language

Machine-level language	Assembly language
The machine-level language comes at the lowest level in the hierarchy, so it has zero abstraction level from the hardware.	The assembly language comes above the machine language means that it has less abstraction level from the hardware.
It cannot be easily understood by humans.	It is easy to read, write, and maintain.
The machine-level language is written in binary digits, i.e., 0 and 1.	The assembly language is written in simple English language, so it is easily understandable by the users.
It does not require any translator as the machine code is directly executed by the	In assembly language, the assembler is used to convert the assembly code into machine code.

computer.	
It is a first-generation programming language.	It is a second-generation programming language.

1.6 High Level Languages

We have talked about programming languages as COBOL, FORTRAN and BASIC. They are called high level programming languages. The program shown below is written in BASIC to obtain the sum of two numbers.

```
LET    X      =    7
LET    Y      =    10
LET    sum    =    X+Y
PRINT SUM
END
```

The time and cost of creating machine and assembly languages was quite high. And this was the prime motivation for the development of high level languages. Because of the difficulty of working with low-level languages, high-level languages were developed to make it easier to write computer programs. High level programming languages create computer programs using instructions that are much easier to understand than machine or assembly language code because you can use words that more clearly describe the task being performed.

When writing a program in a high-level language, then the whole attention needs to be paid to the logic of the problem. A compiler is required to translate a high-level language into a low-level language.



High-level languages include FORTRAN, COBOL, BASIC, PASCAL, C, C++ and JAVA.

Advantages of a high-level language

1. Readability: Programs written in these languages are more readable than assembly and machine language.
2. Portability: Programs could be run on different machines with little or no change. We can, therefore, exchange software leading to creation of program libraries.
3. Easy debugging: Errors could easily be removed (debugged).
4. Easy Software development: Software could easily be developed. Commands of programming language are similar to natural languages (ENGLISH).

Differences between Low-Level language and High-Level language

Low-level language	High-level language
It is a machine-friendly language, i.e., the computer understands the machine language,	It is a user-friendly language as this language is written in simple English words, which can

which is represented in 0 or 1.	be easily understood by humans.
The low-level language takes more time to execute.	It executes at a faster pace.
It requires the assembler to convert the assembly code into machine code.	It requires the compiler to convert the high-level language instructions into machine code.
The machine code cannot run on all machines, so it is not a portable language.	The high-level code can run all the platforms, so it is a portable language.
It is memory efficient.	It is less memory efficient.
Debugging and maintenance are not easier in a low-level language.	Debugging and maintenance are easier in a high-level language.

1.7 Introduction to C Programming

The programming language C was originally developed by Dennis Ritchie of Bell Laboratories and was designed to run on a PDP-11 with a UNIX operating system. Although it was originally intended to run under UNIX, there has been a great interest in running it under the MS-DOS operating system on the IBM PC and compatibles. It is an excellent language for this environment because of the simplicity of expression, the compactness of the code, and the wide range of applicability. Also, due to the simplicity and ease of writing a C compiler, it is usually the first high level language available on any new computer, including microcomputers, minicomputers, and mainframes. It allows the programmer a wide range of operations from high level down to a very low level, approaching the level of assembly language. There seems to be no limit to the flexibility available.

1.8 Origin and Development of C Language

C is a general-purpose, structured programming language. Structured Languages have a characteristic program structure and associated set of static scope rules. C was originated in Bell Telephone Laboratories presently known as AT & T Bell Laboratories by Dennis Ritchie in 1970. The Kernighan and Ritchie description is commonly referred to as "K&R C". Following the publication of the K & R description, computer professionals, impressed with C's many desirable features, began to promote the use of the language. Since 1980's, the popularity of C has become widespread. The American National Standards Institute (ANSI) proposed a standardized definition of the C language (ANSI committee X3J11). Most commercial C compilers and interpreters are expected to adopt the ANSI standard.

C has the feature of high level programming language as well as the low-level programming. It works as a bridging gap between machine language and the more conventional high-level languages. This feature of C Language made it most popular for system programming as well as application programming.

1.9 Applications of C Language

Mainly C Language is used for Develop Desktop application and system software. Some application of C language are given below.

- C programming language can be used to design the system software like operating system and Compiler.
- To develop application software like database and spread sheets.
- For Develop Graphical related application like computer and mobile games.
- To evaluate any kind of mathematical equation use c language.
- C programming language can be used to design the compilers.
- UNIX Kernel is completely developed in C Language.
- For Creating Compilers of different Languages which can take input from other language and convert it into lower level machine dependent language.
- C programming language can be used to design Operating System.

- C programming language can be used to design Network Devices.
- To design GUI Applications. Adobe Photoshop, one of the most popularly used photo editors since olden times, was created with the help of C.

1.10 Evolution of C

By the late fifties, there were many computer languages into existence. However, none of them were general purpose. They served better in a particular type of programming application more than others. Thus, while FORTRAN was more suited for engineering programming, COBOL was better for business programming. At this stage people started thinking that instead of learning so many languages for different programming purposes, why not have a single computer language that can be used for programming any type of application.

In 1960, to this end, an international committee was constituted which came out with a language named ALGOL-60. This language could not become popular because it was too general and highly abstract.

In 1963, a modified ALGOL-60 by reducing its generality and abstractness, a new language, CPL (Combined Programming Language) was developed at Cambridge University. CPL, too turned out to be very big and difficult to learn.

In 1967, Martin Richards, at Cambridge University, stripped down some of the complexities from CPL retaining useful features and created BCPL (Basic CPL). Very soon it was realized that BCPL was too specific and much too less powerful.

In 1970, Ken Thompson, at AT&T labs. Developed a language known by the name B as another simplification to CPL. B, too, like its predecessors, turned out to be very specific and limited in application.

In 1972, Ritchie, at AT&T, took the best of the two BCPL and B, and developed the language C. C was truly a general purpose language, easy to learn and very powerful.

In 1972, Ritchie, at AT&T, took the best of the two BCPL and B, and developed the language C. C was truly a general purpose language, easy to learn and very powerful.



Give two examples of high level languages.

1.11 Compiler and Interpreter

Note that the only language a digital computer understands is binary coded instructions. Even the above implementation will not execute on a computer without further translation into binary (machine) code. This translation is not done manually, however. There are programs available to do this job. These translation programs are called compilers and interpreters.

Compilers and interpreters are programs that take a program written in a language as input and translate it into machine language. Thus a program that translates a C program into machine code is called C compiler; BASIC program into machine code is called a BASIC compiler and so on.

Compilers and interpreters are programs that take a program written in a language as input and translate it into machine language. Thus a program that translates a C program into machine code is called C compiler; BASIC program into machine code is called a BASIC compiler and so on.

A number of different compilers are available these days for C language. GCC, ANSI, Borland C, Turbo C, etc. are only few of the popular C compilers. As a matter of fact, these software tools are little more than just compiler. They provide a complete environment for C program development. They include, among others, an editor to allow Program writing, a Compiler for compilation of the same, a debugger for debugging/testing the program, and so forth. Such tools are referred to as IDE (Integrated Development Environment) or SDK (Software Development Kit).



Code blocks is an IDE for running C and C++ programs on different operating systems like Windows, Linux and Mac OS.

1.12 Program Development in C

The development of a "C" program involves the use of the following programs in the order of their usage.

Editor

This program is used for writing the Source Code, the first thing that any programmer writing a program in any language would be doing.

Debugger

This program helps us identify syntax errors in the source code.

Pre Processor

There are certain special instructions within the source code identified by the # symbol that are carried on by a special program called a preprocessor.

Compiler

The process of converting the C source code to machine code and is done by a program called Compiler.

Linker

The machine code relating to the source code you have written is combined with some other machine code to derive the complete program in an executable file. This is done by a program called the linker.

1.13 Writing a C Program

The following rules are applicable to all C-statements:

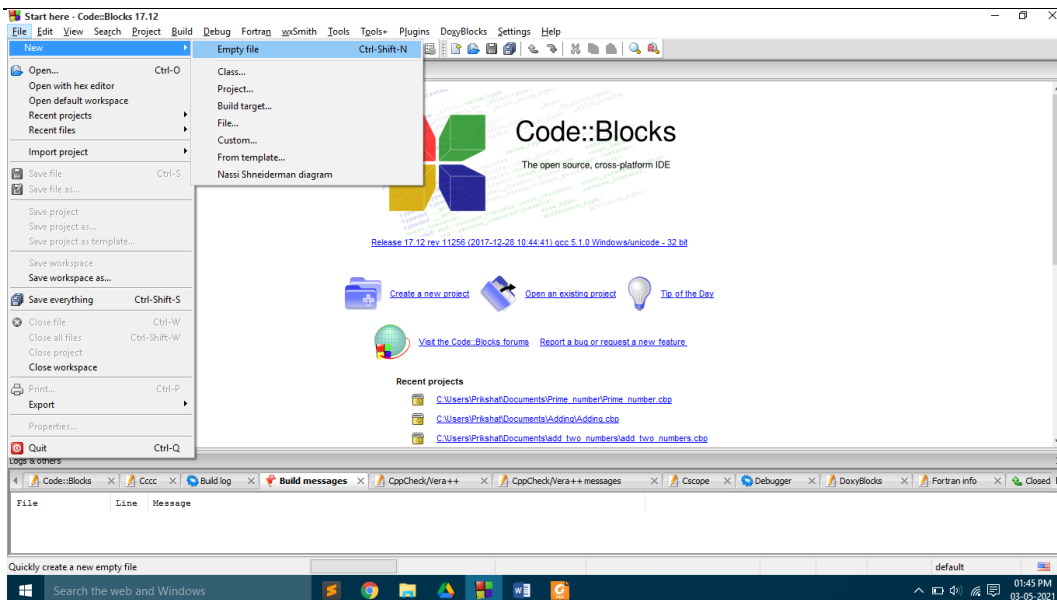
1. Blank spaces may be inserted between two words to improve the readability of the statement. However, no blank space is allowed within a word.
2. Most of the C-compilers are case-sensitive, and hence statements are entered in small case letters.
3. C has no specific rules about the position at which different parts of a statements be written. Not only can a C statement be written anywhere in a line, it can also be split over multiple lines. That is why it is called free-format language.
4. A C-statement ends with a semi-colon (;)
5. Every C program contains one main() function.

C Program Code

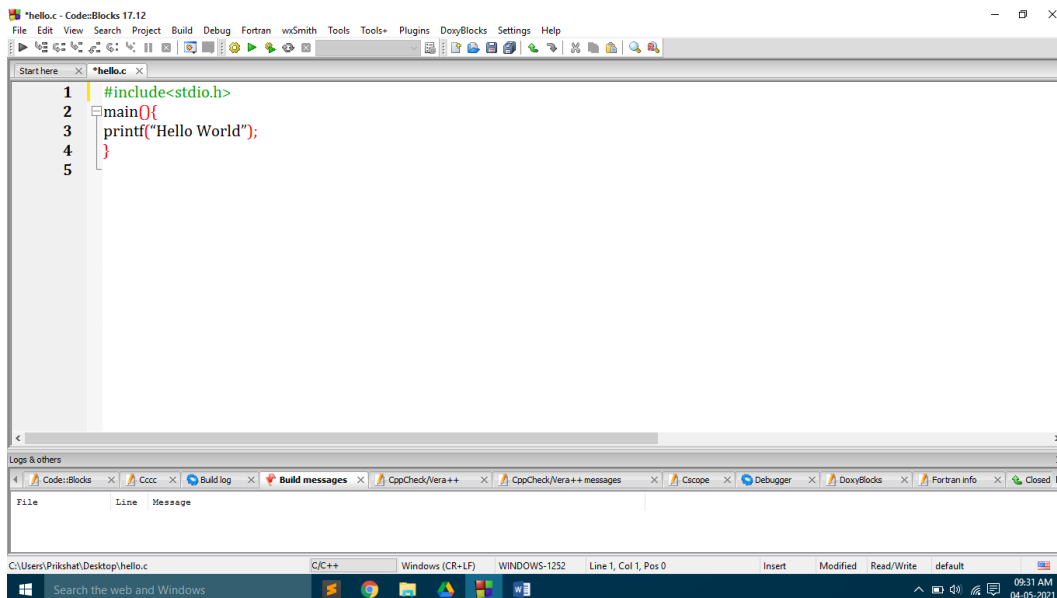
```
#include<stdio.h>
main(){
printf("Hello World");
}
```

Creating and Compiling a C Program

Creating a compiling a C program on operating system use compiler and an integrated development environment. Code blocks is used for create and execute program of C language. File name is hello.c and save in windows operating system using code blocks IDE.



Click on empty file link and save that file with name hello.c and write code.



After write code in code blocks,

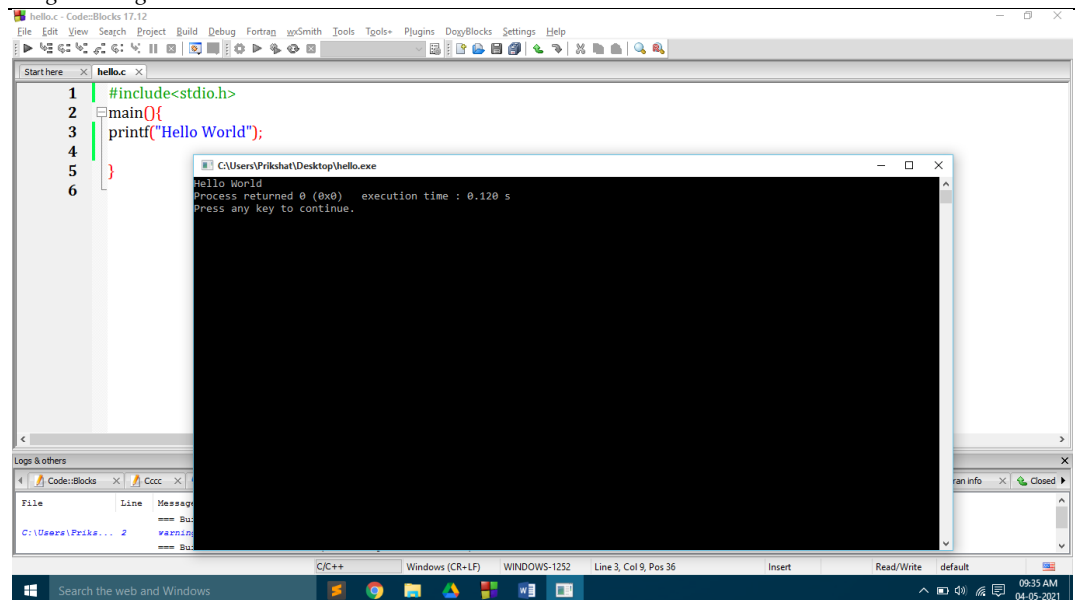
Gcc compiler is used for compiling code using code blocks

For compile press CTRL+F9 or click on build option and click on build

To run C program in code blocks after write code press first compile program than run (CTRL+F10).

Output will be

Programming in C



1.14 C Character set

Like each other language, 'C' additionally has its own character set. A program is a bunch of directions that, when executed, produce a yield. The information that is prepared by a program comprises of different characters and images. The yield produced is additionally a mix of characters and images.

A character set in 'C' is divided into

- Letters
- Numbers
- Special characters
- White spaces (blank spaces)

Letters

- Uppercase characters (A-Z)
- Lowercase characters (a-z)

Numbers

- All the digits from 0 to 9

White spaces

- Blank space
- New line
- Carriage return
- Horizontal tab

Special characters

- Special characters in 'C' are shown in the given table,

, (comma)	{ (opening curly bracket)
. (period)	} (closing curly bracket)
; (semi-colon)	[(left bracket)
: (colon)	] (right bracket)
? (question mark)	((opening left parenthesis)
' (apostrophe)	) (closing right parenthesis)
" (double quotation mark)	& (ampersand)
! (exclamation mark)	^ (caret)
(vertical bar)	+ (addition)
/ (forward slash)	- (subtraction)
\ (backward slash)	* (multiplication)
~ (tilde)	/ (division)
_ (underscore)	> (greater than or closing angle bracket)
\$ (dollar sign)	< (less than or opening angle bracket)
% (percentage sign)	# (hash sign)
, (comma)	{ (opening curly bracket)

1.15 Identifiers and keywords

Keywords have fixed meanings, and the meaning cannot be changed. They act as a building block of a 'C' program. There are a total of 32 keywords in 'C'. Keywords are written in lowercase letters.

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	short	float	unsigned
continue	for	signed	void

An identifier is nothing but a name assigned to an element in a program. Example, name of a variable, function, etc.

Summary

- A computer programming language consists of a set of symbols and characters, words, and grammar rules that permit people to construct instructions in the format that can be interpreted by the computer system. Computer Programming is the art of making a computer do what you want it to do.
- Machine language, or machine code, is a low-level language comprised of binary digits (ones and zeros).
- Assembly languages are also known as second generation languages. These languages substitute alphabetic symbols for the binary codes of machine language.
- High-level languages include FORTRAN, COBOL, BASIC, PASCAL, C, C++ and JAVA.
- C is a general-purpose, structured programming language. Structured Languages have a characteristic program structure and associated set of static scope rules. C was originated in Bell Telephone Laboratories presently known as AT & T Bell Laboratories by Dennis Ritchie in 1970.
- To develop application software like database and spread sheets.
- For Develop Graphical related application like computer and mobile games.
- Keywords have fixed meanings, and the meaning cannot be changed. They act as a building block of a 'C' program. There are a total of 32 keywords in 'C'.
- Code blocks is an IDE for running C and C++ programs on different operating systems like Windows, Linux and Mac OS.
- An identifier is nothing but a name assigned to an element in a program. Example, name of a variable, function, etc.

Keywords

Programming: Computer programming is the process of designing and building an executable computer program to accomplish a specific computing result or to perform a specific task.

Machine Level Language: Machine language, or machine code, is a low-level language comprised of binary digits (ones and zeros).

Assembly Level Language: Assembly language is a low-level programming language.

High Level Language: high-level programming language is a programming language with strong abstraction from the details of the computer.

Self-Assessment

1. Which of the following language instructions execute very fast?
 - A. Machine language Instructions
 - B. Hardwired Instructions
 - C. Procedural language Instructions
 - D. Assembly language Instructions
2. Program written in _____ language is difficult to understand.
 - A. Procedural language
 - B. Machine Language
 - C. Assembly Language
 - D. High Level Language.
3. Which of the following is part of computer system.
 - A. Hardware
 - B. Programming
 - C. Software
 - D. Hardware & Software
4. _____ is a computer language.
 - A. Binary Language
 - B. Low Level Language
 - C. High Level Language
 - D. All of the above
5. Compiling is a process.
 - A. To convert source code to machine independent code.
 - B. To make program.
 - C. To complete coding.
 - D. All of the above.
6. Who developed C language?
 - A. Dennis M. Ritchie
 - B. Steve Jobs
 - C. Steve Smith
 - D. James Gosling

7. What will this program print?

```
#include<stdio.h>
int main()
{
/* my first program in C */
printf("Hello, World! \n");
return 0;
}
```

- A. Hello, World!
- B. Hello, Programming
- C. Error Message

- D. All of the above
8. Which of the following is an extension of C language code?
- A. .c
 - B. .x
 - C. .z
 - D. None of the above
9. Which of the following language is used to write Unix kernel?
- A. C#
 - B. Java
 - C. C
 - D. SQL
10. Which of the following statement is true about comments?
- A. Comments are C++ statements.
 - B. Comments can be inserted anywhere in C Program.
 - C. Comments are required in every program.
 - D. All of the above.
11. Which is not keyword in C language?
- A. complex
 - B. near
 - C. typedef
 - D. const
12. We can use keywords as Identifier.
- A. True
 - B. False
13. Which of the following statement is true about identifier?
- A. An identifier is used to identify elements of a program
 - B. An identifier acts as comment
 - C. An identifier is similar to keywords
 - D. All of the above
14. Which of the following statement is true about keyword?
- A. Keywords are same as identifier
 - B. Keywords are not part of C language
 - C. Reserve words in C language are known as keywords.
 - D. All of the above
15. C character set includes
- A. Letters & numbers
 - B. Letters, numbers and white spaces
 - C. Special characters only
 - D. Letters, numbers, white spaces and special characters

Answer for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. A | 2. B | 3. D | 4. A | 5. A |
| 6. A | 7. A | 8. A | 9. C | 10. B |
| 11. B | 12. B | 13. A | 14. C | 15. D |

**Further Readings**

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008.

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi.

Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi.

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C

**Web Links**

<https://www.tutorialspoint.com/index.htm>

www.webopedia.com

www.web-source.net

Unit 02: Data Types

CONTENTS

Objectives

Introduction

2.1 Constants in C

2.2 Data Types

2.3 Variables

Summary

Keywords

Self Assessment

Answers for Self Assessment

Further Readings

Objectives

After studying this unit, you will be able to:

- Understand constant and variables
- Explain data type in C
- Describe C program structure
- Describe how to declare variables in C

Introduction

Data items (variables and constants) can be manipulated arithmetically in a C programme using operators. An operator is a symbol that instructs the machine to perform mathematical or logical operations on data held in variables. Operands are the variables that are being worked on.

2.1 Constants in C

A constant is a non-changing token with a fixed value. It can be stored in a place in the computer's memory and accessed using that memory address. In C, constants are divided into four categories: integer constants, floating-point constants, character constants, and string constants. Constants can be present in composite forms.

Integer and floating-point constants represent numbers. They are often referred to collectively as numeric-type constants.

C imposes the following rules while creating a numeric constant type data item:

1. Commas and blank spaces cannot be included within the constants.
2. The constant can be preceded by a minus (-) sign if desired.
3. Value of a constant cannot exceed specified maximum and minimum bounds. For each type of constant, these bounds will vary from one C-compiler to another.

Constants are the fixed values that remain unchanged during the execution of a program and are used in assignment statements. Constants can also be stored in variables.

The declaration for a constant in C language takes the following form:

<Data type > <variable_name> = <value>



Example: float pi = 3.14

This declaration defines a constant named pi whose value remains 22/7 throughout the program in which it is defined.

C language facilitates five different types of constants.

1. Character
2. Integer
3. Real
4. String
5. Logical

Character Constants

A character constant consists of a single character, single digit, or a single special symbol enclosed with in a pair of single inverted commas. The maximum length of a character constant is one character.

Example: 'a' is a character constant

Example: 'd' is a character constant

Example: 'P' is a character constant

Example: '7' is a character constant

Example: '*' is a character constant

Integer Constants

An integer constant refers to a sequence of digits and has a numeric value. There are three types of integers in C language: decimal, octal and hexadecimal.

Decimal integers	1, 56, 7657, -34 etc.
Octal integers	076, -076, 05 etc. (preceded by zero, 0)
Hexadecimal integers	0x56, -0x5D etc. (preceded by zero, 0x)

No commas or blanks are allowed in integer constants.

Real or Floating Point Constants

A number with a decimal point and an optional preceding sign represents a real constant.



Example: 34.8, -655.33, .2, -.56, 7.



Note: that 8 is an integer while 8. or 8.0 is real.

Another notation (called scientific notation) for real constants consists of three parts:

1. A sign (+ or 0) preceding the number portion (optional).
2. A number portion.
3. An exponent portion following the number portion (optional). It starts with E or e followed by an integer. This may or may not be preceded by a sign.



Example:

Valid representations	Invalid Representations
+ . 72	12 (no decimal)
+ 72.	7.6 E + 2.2 (non integer exponent)
+ 7.6 E + 2	1.2 E9229892 (very large exponent)
24.4 e - 5	

String Constants

A string constant is a sequence of one or more characters enclosed within a pair of double quotes (""). If a single character is enclosed within a pair of double quotes, it will also be interpreted as a string constant and not a character constant.



Example:

1. " Hello World"
2. "A"

Actually, a string is an array of characters terminated by a NULL character. Thus, "a" is a string consisting of two characters, viz. 'a' and NULL('\0').

Logical Constants

The value of a logical constant may be true or false. In C, a non-zero value is considered true, while 0 is considered false.



Task: Point out the error, if any, in this C statement:

```
int n=512.225;
```

2.2 Data Types

One of the factors that determines the strength of a programming language is the number of different types of data that it can handle. In this regard, the C programming language is extremely strong. In a C programme, almost every form of data can be interpreted and manipulated.

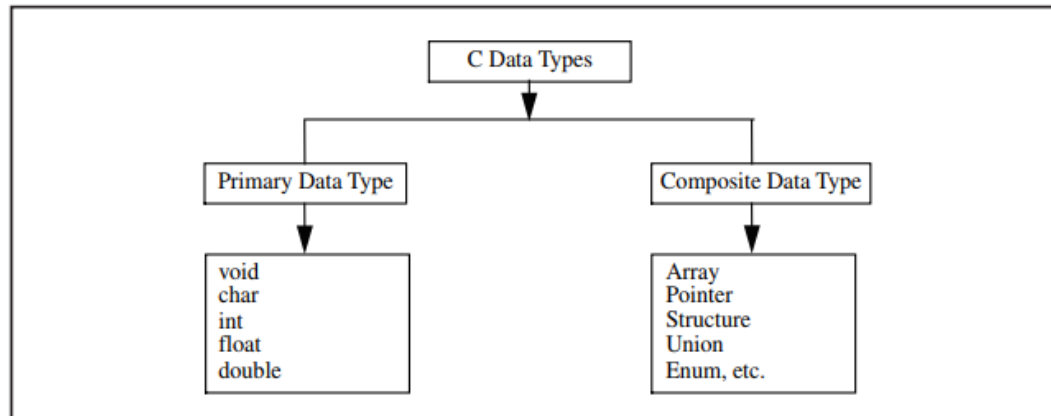
Inside a digital computer, at the lowest level, all data and instructions are stored using only binary digits (0 and 1). Thus, decimal number 65 is stored as its binary equivalent: 0100 0001. Also the character "A" is stored, as binary equivalent of 65(A's ASCII): 0100 0001. Both the stored values are same but represent different type of values. How's that?

Actually, the interpretation of a stored value depends on the type of the variable in which the value is stored even if it is just 0100 0001 as long as it is stored on the secondary storage device. Thus, if 0100 0001 is stored in an integer type variable, it will be interpreted to have integer value 65, whereas, if it is stored in character type of variable, it will represent "A".

Therefore, the way a value stored in a variable is interpreted is known as its data type. In other words, data type of a variable is the type of data it can store.

Every computer language has its own set of data types it supports. Also, the size of the data types (number of bytes necessary to store the value) varies from language to language. Besides, it is also hardware platform dependent.

C has a rich set of data types that is capable of catering to all the programming requirements of an application. The C-data types may be classified into two categories: Primary and Composite data types as shown in figure.



C has two distinct categories of data types – primary, and composite. Primary data types are the ones that are in-built with C language while composite data types allow the programmers to create their own data types.

There are five primary data types in C language.

1. **char**: stores a single character belonging to the defined character set of C language.



Example: char a;

2. **int**: stores signed integers. e.g., positive or negative integers.



Example: int b;

3. **float**: stores real numbers with single precision (precision of six digits after decimal points)



Example: float c;

4. **double**: stores real numbers with double precision, i.e., twice the storage space required by float.



Example: double d;

5. **void**: specify no values.

The following table shows the meaning and storage spaces required by various primary data types.

Data Type	Meaning	Storage Space	Format	Range of Values
char	A character	1 byte	%c	ASCII character set
int	An integer	2 bytes	%d	– 32768 to +32767
float	A single precision floating point number	4 bytes	%f	– 3.4*10 ³⁸ to + 3.4*10 ³⁸

double	A double precision floating point number	8 bytes	%lf	– $1.7 \times 10_{308}$ to $+1.7*10_{308}$
void	valueless or empty	0 byte	–	–

Additional Data Types

Primary C data types can come in a variety of sizes, allowing them to store a wide range of values. This is implied in a programme by appending a keyword (data type qualifier) before the data type.

For example, data type qualifiers for the int basic type include short, long, signed, and unsigned. In C, an integer data type can be specified as a short int, int, unsigned int, or long int. Implementation determines the set of values and size of these eligible data-types. Short, on the other hand, is smaller than or equal to int, which is smaller than long. Since it does not store negative integers, an unsigned int has a wider range.

Also known as derived data types, composite data types are derived from the basic data types. They are five in number.

1. **Array:** Sequence of objects, all of which are of same types and have same name.



Example: int num [5];

Reserves a sequence of five locations of 2 bytes, each, for storing integers num[0], num[1], num[2], num[3] and num[4].

2. **Pointer:** Used to store the address of any memory location.



Example: int *ptr.



Note: - Asterisk (*) symbol is used to declare pointer variable.

3. **Structure:** Collection of variables of different types.



Example: A structure of employee's data, i.e., name, age and salary.

4. **Union:** Collection of variables of different types sharing common memory space.
5. **Enumerated:** Its members are the constants that are written as identifiers though data type they have signed integer values. These constants represent values that can be assigned to corresponding enumeration variables.

Enumeration may be defined as:

```
enum tag { member1, member2 .... member n};
```

E.g.: enum colors {red, green, blue, cyan};

```
colors foreground, background;
```

In the first line, an enumeration named "colors" which may have any one of the four colors defined in the curly braces. In the second line, variables of the enumerated data type "colors" are declared.



Notes: Data types always used on the basis of their length value suggest the range of float data type.

2.3 Variables

A variable is an object whose value can change while a programme is running. A variable is a symbolic representation of the address in memory space where values can be stored, accessed, and updated. Each variable has a memory location or address assigned to it, and the value of that variable is stored there.

Each variable has its own name, data type, size, and value. All variables must have a type so that the compiler can record all necessary information about them, generate the appropriate code during translation, and allocate the necessary memory space.

Every programming language has its own set of rules that must be observed while writing the names of variables. If the rules are not followed, the compiler reports compilation error. Rules for Constructing Variable Name in C language are listed below:

1. Variable name may be a combination of alphabets, digits or underscores. Sometimes, an additional constraint on the number of characters in the name is imposed by compilers in which case its length should not exceed 8 characters.
2. First character must be an alphabet or an underscore (_).
3. No commas or blank spaces are allowed in a variable name.
4. Among the special symbols, only underscore can be used in a variable name. E.g.: emp_age, item_4, etc.
5. No word, having a reserved meaning in C can be used for variable name



Notes: C language is a case-sensitive language which differentiates between lower case and upper case. Thus, JOB is different from Job, jOB, JoB. Although any word complying with the rules cited above can be used as the variable name, it is advised that you create variable names that have some meaning. Thus, you may chose sum as the variable name for storing sum of numbers rather than choosing X.

Declaration of Variables

C language is strongly typed language, meaning that all the variables must be declared before their use. Declaration does two things:

1. It tells the compiler what the variable name is.
2. It specifies what type of data the variable will hold

In C language, a variable declaration has the form:

<Type-specifier> <comma-separated-list-of-variables>;

Here <type-specifier> is one of the valid data types (e.g. int, float, char, etc.). List-of-variables is a comma-separated list of identifiers representing the program variables.



Example: `int i, j, k; //creates integer variables i,j and K`

Once variable has been declared in the above manner, the compiler creates a space in the memory and attaches the given name to this space. This variable can now be used in the program.

A value is stored in a variable using assignment operation. Assignment is of the form:

<Variable-name> = <value>;

Obviously, before assignment, the variable must be declared.



Example:

```
int a , b;
a = 100;
b = 20;
```

C also allows assignment of a value to a variable at the time of declaration. It takes the following form:

<Type-specifier> <variable_name> = <value>;

e.g. : int I = 5;



Notes: This declaration not only creates an interior type variable I, but also stores a value 5 at the same time.

The process of assigning initial values to the variable is known as initialization. More than one variable can be initialized in one statement using multiple assignment operators.



Example:

```
1. int i = 10, j = 15;
2. int j , m ;
   j = m =2 ;
```

However, there is an exception worth noting. Consider the following example:

```
int i, j = 2, k;
```

The assignments will be made as follows:

```
i = 0
```

```
j = 2
```

```
k = 1063 (a garbage value, uninitialized)
```



Did you know?

Allocating a variable implies reserving some memory for that variable. In some programming languages (like C) if a variable is allocated but not assigned, it is said to have a "garbage value".

Let us consider some of programming examples to illustrate the matter further.

```
/* Example of assignments */
/* declaration */
int a1, b1;
/* declaration & assignment */
int var = 5;
```

Programming In C

```
int a, b = 6, c;  
/* declaration & multiple assignment */  
int p, q, r, s;  
p = q = r = s = 5;  
}
```

values stored in various variables are:

```
var = 5  
a = 0, b = 6  
c = garbage value  
p = 5, q = 5, r = 5, s = 5
```



Notes: Write a program in C to show the variable declaration.



Example:

Write a program to find sum of two numbers.

Solution : - #include<stdio.h>

```
int main(){  
int num1,num2,sum;  
num1=100;  
num2=300;  
sum=num1+num2;  
printf("Sum of %d and %d is = %d",num1,num2,sum);  
return 0;  
}
```

Output:-

```
Sum of 100 and 300 is = 400  
Process returned 0 (0x0)   execution time : 0.056 s  
Press any key to continue.
```

Summary

- A constant is a value that doesn't change throughout the execution of a program.
- A variable is an identifier which is used to store a value.

- There are four commonly used data types such as int, float, char and a void.
- Each data type differs in size and range from one another.
- C is a programming language. It has letters, words, sentences and a well-defined grammar. Character set of a language is set of all the symbols used to write a program in that language.
- Words formed in language C are termed as identifiers. C identifiers can be of two types - keywords or reserved words, and user-defined identifiers. Every language contains certain words that have specific predefined meaning associated with them. Such words are known as Keywords.
- A token is a group of characters separated from other group of characters by one or more white space. A constant is a token with fixed value that does not change.
- A variable is an entity whose value can change during program execution. A variable can be thought of as a symbolic representation of address of the memory space where values can be stored, accessed and changed.
- C language is a case-sensitive language which differentiates between lower case and upper case. C language is strongly typed language, meaning that all the variables must be declared before their use. C has two distinct categories of data types – primary, and composite. Primary data types are the ones that are in-built with C language while composite data types allow the programmers to create their own data types.

Keywords

Constant: A named data item whose value does not change throughout the execution of the Program

Variable: A named location in the memory that can store a value of specified type.

Data type: A data type specifies the type of data that a variable can store such as integer, floating, character, etc.

Character constant: A "character constant" is formed by enclosing a single character from the representable character set within single quotation marks (' ').

White space: The characters that separate one identifier from the other like space, tab, carriage return, new line etc.

Self Assessment

1. Which of the following is true for variable names in C?
 - A. Variable can be of any length
 - B. They can contain alphanumeric characters as well as special characters
 - C. Reserved Word can be used as variable name
 - D. Variable names cannot start with a digit
2. What are the entities whose values can be changed called?
 - A. Constants
 - B. Variables
 - C. Modules
 - D. Tokens
3. Which of the following is not a constant type in C language?

- A. Real
 - B. Integer
 - C. Main
 - D. Character
4. Which is not a valid C variable name?
- A. int number;
 - B. float rate;
 - C. int variable_count;
 - D. int \$reg_no;
5. The format identifier '%d' is used for _____ data type?
- A. Char
 - B. Int
 - C. Double
 - D. Float
6. The format identifier '%f' is used for _____ data type?
- A. Char
 - B. float
 - C. Double
 - D. Int
7. Constant is _____.
- A. A value that is same as keyword
 - B. A value that doesn't change throughout the execution of a program.
 - C. A value that change throughout the execution of a program.
 - D. All of the above
8. Which of the following is not a valid format specifier?
- A. %d
 - B. %\d
 - C. %f
 - D. %c
9. Which of the following is type of variable in C language?
- A. Local Variable
 - B. Global Variable
 - C. All of the above
 - D. None of the above
10. Which is the only function all C programs must contain?

- A. start()
- B. system()
- C. main()
- D. printf()

11. Which of the following special symbol allowed in a variable name?

- A. * (asterisk)
- B. _ (underscore)
- C. | (pipeline)
- D. - (hyphen)

12. What are the different types of real data type in C?

- A. float, double
- B. short int, double, long int
- C. float, double, long double
- D. double, long int, float

13. What will this program print?

```
#include<stdio.h>
int main()
{
int a,b,sum;
a=100,b=200;
sum=a+b;
/* my first program in C */
printf(sum);
return 0;
}
```

- A. Error
- B. sum
- C. 300
- D. All of the above

14. Which is false?

- A. Global constant variables are initialized to zero
- B. const keyword is used to define constant values
- C. Constant variables need not be defined as they are declared and can be defined later
- D. You cannot reassign a value to a constant variable

15. Which of the following symbol is used in format specifier?

- A. !
- B. @

- C. \$
- D. %

Answers for Self Assessment

- | | | | | |
|-------|-------|-------|-------|-------|
| 1. D | 2. B | 3. C | 4. D | 5. B |
| 6. B | 7. B | 8. B | 9. C | 10. C |
| 11. B | 12. C | 13. A | 14. C | 15. D |



Further Readings

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008.

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi.

Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi.

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

Yashvant Kanetkar, Let us C



Web Links

<https://www.tutorialspoint.com/index.htm>

www.webopedia.com

www.web-source.net

Unit 03: Input / Output in C

CONTENTS

Objectives

Introduction

3.1 Input/output Function

3.2 Formatted Input/Formatted Output

3.3 printf()

3.4 Escape Sequences

3.5 scanf()

3.6 sign (*)

3.7 Reading and Writing a Character

3.8 gets() and puts() Functions

Summary

Keywords

Self-Assessment

Answer for Self Assessment

Review Questions

Further Readings

Objectives

After studying this unit, you will be able to:

- Describe formatted input/output
- Explain the input/output function
- Know how to read and write a character

Introduction

If a user is unable to communicate with a computer programme, it is unlikely that the programme would be useful. The software must read input values from the user console and generate the expected useful output to the user in most programming assignments.

C is unique in that, unlike other high-level languages, it does not have any built-in I/O statements in its syntax. We'll look at how C handles different I/O operations in this unit. We'll also hear about the standard library functions in C, as well as the header files that go with them.. This unit is not intended to be a complete treatment of these topics, but it provides enough information so that you can start writing real programs.

3.1 Input/output Function

Data is transferred to peripheral devices such as a computer, keyboard, printer, or secondary storage through I/O operations. Since C has no built-in support for receiving data from input devices (such as a keyboard) or sending data to output devices (such as a monitor), all I/O operations must depend on library functions like printf (). A library is a set of predefined functions contained in one or more files. The creators of the C compiler created a library named "C standard library" that contains many standard I/O functions.

Depending on the compiler and necessary functions, various methods may be used to access these library functions from the standard library. Some compilers scan libraries for named functions

Programming In C

automatically. Most of the time, the programmer must specifically state the library file name during the linking process, resulting in the executable programme only containing the necessary functions.

These functions are stored in a special file with the extension.h (such as stdio.h), which is referred to as a header file. The #include compiler directive can be used to insert a header file into a C programme file, as shown below.

```
#include<stdio.h>
```

The #include directive tells the compiler to read the stdio.h (standard input output header file) file and substitute this line with the contents of that file. Similarly, in order to use other predefined functions, their respective header files must be included in a programme such that the function declaration can be accessed by the programme.

C has a rich set of standard I/O library functions. However, these I/O functions are not the part of C's formal definition. C's standard library function for I/O can be broadly divided into the following categories:

1. Port I/O functions
2. Disk I/O function
3. Console I/O function

The Port I/O function handles various I/O operators on various ports, such as a mouse port or a printer port. The scope of this text does not include a thorough examination of the port I/O functions. The disc I/O functions are used to manipulate files on secondary storage devices such as a floppy disc or a hard disc. Since files are stored in secondary storage, such as discs, disc I/O functions are nothing more than file handling functions.

The term "console" refers to the regular input and output devices in its broadest sense. These basic input or output devices, which are also known as the keyboard and display by default, are dealt with by console I/O functions. These functions take keyboard input and display the results on the computer.

C takes all input and output as stream of characters. A stream is nothing,

but a series of bytes. C language treats all streams equally i.e., whether a program gets input from the keyboard, a disk file or a modem, it considers it as only a stream of characters.

Different streams are used to represent different kinds of data flow. In C, there are three streams associated with console I/O operations.

1. **stdin**: A stream that supplies data to the program i.e., standard input, usually from the keyboard.
2. **stdout**: A stream that receives data from the program i.e., standard output; usually to the monitor.
3. **stderr**: A stream used to keep error messages separate from program's output i.e., standard error; usually points to your terminal screen.

For conducting console I/O operations, C offers a number of functions. These functions allow information to be transferred between the computer's standard input and output devices (i.e., keyboard and monitor). Only a few of them allow you to format input and output operations. Some of them, on the other hand, do not allow you to monitor the format of I/O operations.

From this aspect, console I/O operations can be further categorized as:

1. Unformatted console I/O functions
2. Formatted console I/O functions

It is important to include the standard I/O library header file in order to use these functions. The declarations for these functions can be found in the stdio.h header file. As a result, before using these console I/O features, always include the header file stdio.h in your C programme.

3.2 Formatted Input/Formatted Output

In this category, we have functions that allow input and output operations to be performed in a fixed format. Formatting of I/O operators deals with some of the following issues like:

1. How much field width is required to display the various values on the monitor?
2. How many decimal places are required to display the fractional part of a real number?

3. Should data values be left aligned or right aligned, and how much?
4. How much space between two data values is to be given?
5. How various type of data i.e. integer, character, and string can be used together I/O operators, etc.

Printf() and scanf() are the two most commonly used functions for formatted I/O. (). The printf() function is used to view formatted data items on a standard output device, such as a monitor, while the scanf() function is used to read formatted data input from a standard input device, such as a keyboard. However, due to their greater complexity, both functions are slower than the previous classes of functions. These functions are specified in the stdio.h header file and return EOF if an error or the end of the file occurs. Let's take a closer look at each of these functions one by one.

3.3 printf()

The printf() is one of the most important and useful functions to display data on monitor. We have seen the use of printf() for printing messages in the various example given previously in this book. For example, the statement

```
printf(" this section will discuss printf() in detail");
```

will simply print this message on the monitor. Besides these text messages, a program frequently requires numeric values and the value of other variables to be displayed on the screen.



Example: In order to print the sum of two numbers say a and b, in a new line along with some identifying text, the printf() will take the following form:

```
printf("\n the sum of %d and %d is %d.", a, b, c);
```

If the value of a and b is 5 and 6 respectively, then the output would be as follows:

The sum of 5 and 6 is 11.

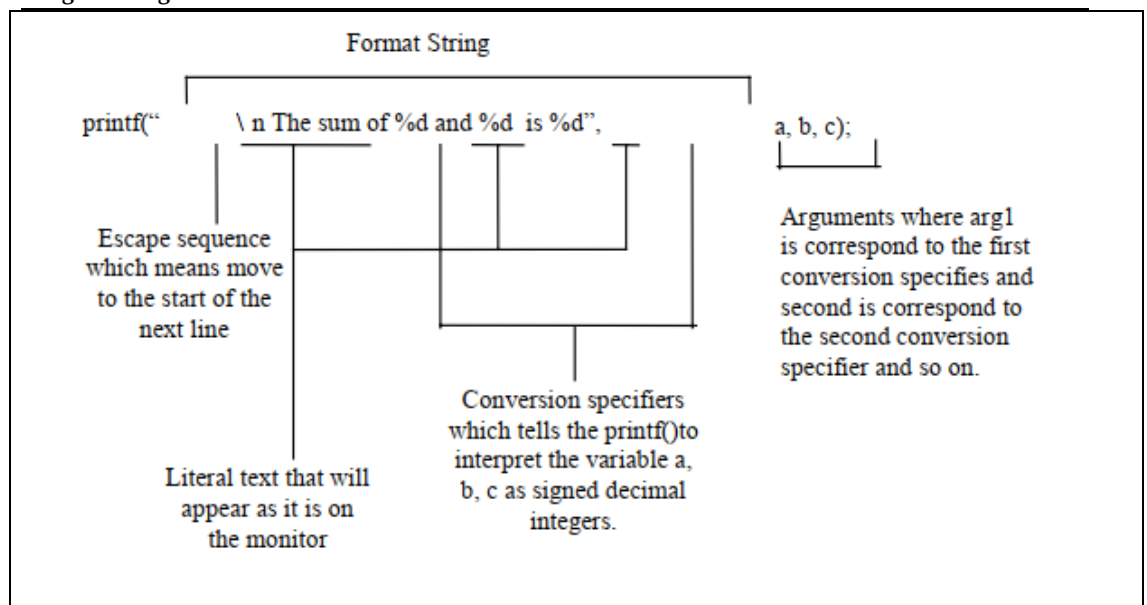
Undoubtedly, a little more complicated than printing a simple message. Before getting the detail of the various sections of this printf(), let's discuss the general format of a printf(), shown below:

```
printf("Format string", arg1, arg2, ..., argn);
```

Where format string refers to a string enclosed in double quotes that contains formatting information and arg1, arg2, ..., argn are arguments (may be constants, variable, or other complex expressions) whose values are formatted and printed according to the specification of the format string.

The format string in a printf() contains the format specifier that defines how the output is formatted. Following are the three possible components of a format string:

1. Literal text that is simply printed as entered in the format string.
2. An escape sequence that begins with a \ (backslash) sign, provides special formatting control.
3. A conversion specifier that begins with a % sign and followed by a single character, that tells printf() how to interpret the arguments being used. To understand the various sections of previously used printf() statement,



The output of given statement if the value of `a` and `b` is 5 and 6 respectively, would be as:

The sum of 5 and 6 is 11

Let's see how this output is evaluated:

The `printf()` function reads the format string from left to right and sends the characters that follow to the standard output unit. It takes action as soon as it encounters the (backslash) (that is, the signal of the existence of an escape sequence). When it comes across the percent (conversion specifier) symbol, it grabs the corresponding argument and prints its value in the format defined. When the closing pair of double quotes is encountered, the process comes to an end.

In our case, the first character after the opening pair of double quote is `\`, followed by a character `n`, so the effect of `\\n` will take place i.e., output of the coming characters will start from the starting of the next line. Output up to this stage appears as:

The sum of

Then comes the character `%` followed by character `d` (that is, the indication to treat the corresponding variable as assigned decimal integer), so it picks up the variable `a` and will print

its value on the screen. At this stage the output will be looking like as:

The sum of 5

In the same manner, this process will go on until there comes an end point of format string. The final output would be appears as:

The sum of 5 and 6 is 11

The following program will help you to understand the concept more closely as it uses the `printf()` statement to print the result of the calculations.

Write a program to print the sum of two numbers.

Program:

```
#include<stdio.h>
```

```
void main()
```

```
{
```

```
int a, b, c;
```

```
a = 5;
```

```
b = 6;
```

```
c = a + b;
```

```
printf("\\n sum = %d", C); /* will print the desired */
```

```
/* result on the monitor */
```

```
}
Output:
Sum = 11
```

Format Specifiers

%d -- displays a decimal (base 10) integer
 %i -- used with other specifiers to indicate a "long"
 %e -- displays a floating point value in exponential notation
 %f -- displays a floating point value
 %c -- displays a single character
 %s -- displays a string of characters

Example: Formatted Output with printf ()

```
Int a;
Float b;
scanf("%d %f", &a, &b);
printf("the value of a= %d and b= %f \n", a, b);
```

3.4 Escape Sequences

Escape sequences, as previously mentioned, are used to control the position of output by shifting the monitor's cursor. Any character preceded by a backslash is meant to be treated as an escape sequence. The backslash informs the compiler that this is a special character constant of different significance than printf ().

Character Constant	Meaning
"X"	The character X
"\X"	The character that follows this is in hexadecimal
"n"	The character n
"\n"	New line

Program shows the usage by some of the frequently used escape sequences

```
#include<stdio.h>
void main()
{
printf("\n 1..\t2..\t3...\n");
printf("The question is, \" said Humpty Dumpty,\" which is to be masterthat\'s all.\");
}
```

Output:

1.. 2.. 3....

"The questions is, "said Humpty Dumpty," which is to be master-that's all".

3.5 scanf()

As previously stated, in order to write an interactive application, certain statements in the programme must be able to collect data from the user. We provided a few functions in this sense,

such as `getchar()`, `gets()`, and `getch()`, among others. A programmer, on the other hand, needs more flexibility in terms of:

1. Read the data from keyboard according to a specified format
2. Instruct the compiler to receive the particular type of value from the keyboard. For instance, integer value or floating-point value.

Instruct the compiler to read the specified number of digits of a given number. Reading mixed data types from the keyboard using single function.

But the use of above mentioned functions is restrict with the character values only. There is a need of more flexible and general function that could address the problems mentioned above.

`scanf()`, the complement of the `printf()`, can actually be used to read the different type of data from the keyboard in a specified format. Due to what, it is referred to as formatted input functions.

Like `printf()`, `scanf()` also uses a format string to describe the format of the input, but with some little variations as given below:

1. It doesn't allow escape sequences in the format string.
2. It requires a special operator `&` called as "address of" to be prefix with the variable identifiers.

So, a `scanf()` takes the following from:

```
scanf(" format string", arg1, arg2,.....argn);`
```

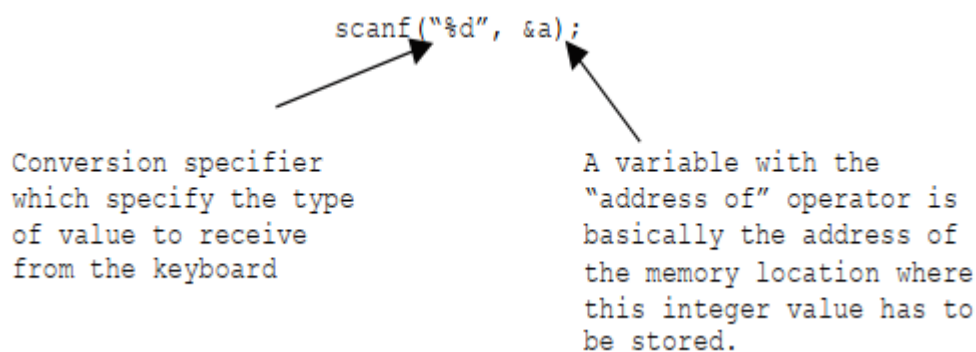
Where format string contains the formatting information by using which the data is to be entered and `arg1, arg2,...., argn` are the arguments (normally variables preceded by an ampersand `&`) specify the address of location where the data is stored. Both the section i.e., format string and arguments (within itself also) must be separated by commas.

The format string in a `scanf()` describe the format of the input and it may contain:

1. Conversion specifiers as in the `printf()` functions.
2. White space characters i.e., tabs, blanks, and newlines.
3. Other characters than white spaces, that are matching characters and asteric

3.6 sign (*)

To have a better understanding of the concept, consider the following statement:



In layman's terms, the preceding statement is an instruction to the compiler to obtain an integer value from the keyboard and store it in a variable called `a`. Where `a` is an integer variable that was previously declared. Following this argument, the value stored in the variable can be used anywhere for any reason. Let's take a closer look at the different components of this sentence.

Conversion Specifier in `scanf()`

As mentioned earlier, a conversion specifier instruct the `scanf()` to convert the input stream of binary data coming from the keyboard in to the data type specified by the conversion character.

For instance, integer in case of `%d` as it utilizes the same conversion specifiers as the `printf()` except one i.e., `[...]`. More than one conversion specifier can be used in a single `scanf()` to read more than one value.

In such a case, corresponding variables each preceded by `&` must include in the same statement.

**Example: The statement.**

```
scanf(" %d %d", &a, &b);
```

Will read two integer values from the keyboard, first value will be assigned to a and second to b.

White Spaces in scanf()

When several variables are entered in a single scanf(), the white space character may be used to distinguish them (i.e., blank space, tabs or new line character). White spaces in the format specifier are not taken into account. As a result, they will be read in the input data but ignored. Example: Consider the following statement once again:

Single space



```
scanf(" %d %d", &a, &b);
```

During runtime, in response to the above statement, you could enter

10 20



single space

As white space can be a tab, so you can also enter as

10 20

tab

A white space can be in the form of newline character, so you could also enter

10 ↵

20

As white spaces are required in input stream, they just can be used to identify the end of each input value.

For real time experience, consider the following program which demonstrates the usage of scanf() to read integer values from the keyboard. This program will accept two numbers from the user and will print their sum on the monitor

**Program: input function**

```
#include<stdio.h>
```

```
int main()
```

```
{
```

```
int var1;
```

```
printf("Enter value of var1");
```

```
scanf("%d",&var1);
```

```
printf("Value of Var1 is = %d",var1);
return 0;
}
```



Program: To find radius of circle

```
#include<stdio.h>

void main()
{
float r, a, c;
const float pi = 3.14;
printf("\n Enter the radius of a circle:");
scanf(" %f", &r);
a = pi * r * r ; /* as r power 2 = r * r */
c = 2 * pi * r;
printf("\n Area = % f", a);
Printf("\n circumference = %f", c);
}
```



Program : lower Case to upper case

```
#include<stdio.h>

void main()
{
charch;
printf("\n Enter any character in lower case:");
scanf("%c", &ch);
printf(" \n The typed character in upper case is %c", ch);
}
```

Output:

Enter any character in lower case: h

The typed character in upper case is: H



If execution is not provided with the proper input, result may be unexpected

3.7 Reading and Writing a Character

Unformatted console I/O functions don't allow input and output to be formatted as per the user requirements. In this category, we have:

1. Character I/O functions
2. String I/O functions

Character I/O functions are functions that programme input/output of one character at a time. Since they deal with individual character values, these are the most basic I/O functions. To enter a character from the keyboard, use the following functions:

1. `getchar()`
2. `getch()`
3. `getche()`

Whereas the output of a character as the monitor, the following functions can be used:

1. `putchar()`
2. `putch()`

Beside these, `getc()` and `putc()` can also be used for one same purpose.

Let us see the working of above-mentioned console I/O functions with the help of programs.

`getchar()` and `putchar()`

`getchar()` function is used for reading a character from the keyboard. The syntax for the `getchar()` function is:

```
Character_variable = getchar(void);
```

Where `character_variable` is any valid C variable name. The word `void` indicates that no argument is needed for calling the function. Following statement reads a character and stores it in variable

`ch`, that is of type `char` obviously.

```
ch = getchar();
```

The `getchar()` waits for the character input until a character is typed at the keyboard. The typed character is echoed to the monitor and before assigning this value to the character variable appeared on the left side, it requires a carriage return (enter key) to be type by the user. `getchar()` function returns a value called EOF (End of File) if an error occurs.

Typically, the value of EOF is 1, though this may vary from compiler to compiler.

The `putchar()` is complementary function of `getchar()`. It is used to display a character on the monitor. The syntax for the `putchar()` function is:

```
putchar (character_variable);
```

Where `character_variable` refers to some previously declared character variable. The following statement displays a character value on the monitor whatever is stored inside `ch` at the current cursor position.

```
putchar(ch);
```

We can also use `putchar()` with character value directly, as shown below.

```
putchar('V');
```

This statement will display the character `V` as the monitor, whereas the statement

```
putchar("\n");
```

would cause the cursor on the screen to move to the beginning of the next line.

This function also returns EOF if there occurs an error.

The following program illustrates the working of these functions. This program will accept a character from the keyboard and will print it on the monitor screen

```
 #include <stdio.h>

void main()
{
    int c;
    printf("Enter a character");
    c = getchar();
```

```
putchar(c);  
}
```

3.8 gets() and puts() Functions

Character I/O functions have a drawback in that they can only handle one character at a time. Strings, on the other hand, are commonly used in real-world programmes. A string is nothing more than a set of characters. String I/O functions are functions that make it easier to move strings between a computer and regular I/O devices. Following function can be used for handling strings I/O:

1. gets()
2. puts()

gets() function is used to accept a string from the keyboard whereas puts() function is used to print a string on the monitor. Besides these I/O functions, C's standard library also provides several functions for various string handling operations. Let's first discuss gets() and puts() in this section.

The gets() function receives a sequence of characters i.e., a string entered at the keyboard and store them in a variable (essentially as Array of type char) mentioned with it.

gets() function

The gets() function allows the user to type a string of characters and then press the enter key. A character list is used to store all of the characters entered by the user. To render the array a string, the null character is inserted. The user will enter space-separated strings using the gets() method. It returns the string that the user typed in. The gets() function is problematic to use because it doesn't search for array bounds and keeps reading characters until the new line (enter) is encountered.

Declaration

```
char[] gets(char[]);
```



Program for gets ()

```
#include<stdio.h>  
  
void main ()  
{  
    char s[25];  
    printf("Enter the string? ");  
    gets(s);  
    printf("You entered %s",s);  
}
```

puts() function

The puts() function looks a lot like the printf() function. The puts() function is used to print the string that was previously read using the gets() or scanf() functions on the console. The integer value returned by the puts() function represents the number of characters printed on the console. Since it prints an extra newline character with the string, which transfers the cursor to a new line on the console, the integer value returned by puts() will always be the number of characters in the string plus 1.

Declaration

```
int puts(char[])
```



```
#include<stdio.h>
#include <string.h>
int main(){
char name[30];
printf("Enter your name: ");
gets(name); //reads string from user
printf("Your name is: ");
puts(name); //displays string
return 0;
}
```



Program For gets() and puts()

```
#include <stdio.h>
void main( )
{
char name[10];
printf("Enter Name");
gets(name);
printf("Entered name is");
puts(name);
return 0;
}
```

Difference between Formatted and Unformatted Functions

- Formatted input and output functions contain format specifier in their syntax.
- Unformatted input and output functions do not contain format specifier in their syntax.
- printf() and scanf() are examples for formatted input and output functions.
- getchar(), gets(), puts(), putchar() etc. are examples of unformatted input output functions.

Function	Purpose
getchar()	Input exactly one character from keyboard
putchar()	Print exactly one character
gets()	Input collection of Characters
puts()	Print collection of Characters

**Program**

```
#include<stdio.h>

void main()
{
    charstr [11]; /* declar a character array str of size 11 */
    printf("\n enter a string (Maximum 10 characters):");
    gets(str); /* will read a string from the keyboard*/
    printf("\n the entered string \n.");
    puts(str); /* will print the under of str on the monitor and */
    print(" /* and advances the cursor to the .....*/ thank you");
}
```

Output:

Enter a string (maximum 10 characters): GoodDay

The entered string is: GoodDay

Thank you

Summary

- I/O operations deal with the transfer of data to peripheral devices such as monitor, key board, printer or secondary storage etc.
- A library is nothing more than one or more files that contain a group of predefined functions. In its most general form the word 'console' refers to the standard input and output devices.
- Unformatted console I/O functions doesn't allow input and output to be formatted as per the user requirements. getchar() function is used for reading a character from the keyboard.
- The putchar() is complementary function of getchar(). It is used to display a character on the monitor.
- The another possible use of getch() is to temporarily halt the execution of a program intentionally. gets() function is used to accept a string from the keyboard whereas puts() function is used to print a string on the monitor.
- Formatted I/O Functions allows input and output operations to be performed in a fixed format.
- The printf() in one of the most important and useful functions to display data on monitor.
- Except for double quotes, escape sequences and conversions specifiers, all characters within a pair of double quotes will be treated as literal text (string context) and will be display as it is a on the monitor.
- Any character that is prefix with a backslash is supposed to be treated as an escape sequence.
- Conversion's specifiers always begin with the percent sign (%) and immediately followed by one or more conversions characters
-

Keywords

printf()

scanf()

getchar()	putchar()
gets()	puts()

Header files: A text file that contains prototype of functions, definitions of constants etc. and which can be included in a C program file to access those functions and constants.

#include compiler directive: This compiler directive instructs the compiler to insert the contents of the specified file in place of this line.

Standard library: A group of in-built functions stored in a single file as a unit

Self-Assessment

1. Which format specifier is used for integer
 - A. % c
 - B. % d
 - C. % f
 - D. % h
2. Which is correct file pointer for standard input
 - A. Stdout
 - B. Stdin
 - C. Stderr
 - D. Stdab
3. Which function is used for output on screen
 - A. Main ()
 - B. Scanf ()
 - C. Printf ()
 - D. Get ()
4. What is extension of header file?
 - A. .c
 - B. .b
 - C. .e
 - D..h
5. main () is a
 - A. Keyword
 - B. Function
 - C. Integer
 - D. Pointer
6. Which format specifier is used for float?
 - A. % c
 - B. % d
 - C. % h
 - D. % f
7. Input is taken using function....
 - A. Printf ()

- B. Fprintf ()
 - C. Scanf ()
 - D. Main ()
8. Chose the odd one.
- A. Put()
 - B. Printf()
 - C. Putchar()
 - D. Scanf()
9. Putchar() is used for..
- A. As input function for string
 - B. display exactly one character as output.
 - C. Display one string on screen
 - D. Input from user
10. Getchar() having similarities with..
- A. Puts ()
 - B. Gets ()
 - C. Main ()
 - D. Putchar ()
11. Default return-type of getchar()?
- A. Int
 - B. float
 - C. char
 - D. reading character doesn't require a return-type
12. What is the return value of putchar()?
- A. EOF if an error occurs
 - B. Nothing
 - C. Both character written & EOF if an error occurs
 - D. All of the above
13. Puts() function is used for
- A. Print collection of Characters
 - B. Input collection of Characters
 - C. Print exactly one character
 - D. All of above
14. Which statement is incorrect
- A. char ch;
 - B. ch = getchar () ;

C. putchar (ch) ;

D. main () :

15. Which one is true about formatted functions?

A. getchar(), gets(), puts(), putchar() etc. are examples of formatted input output functions.

B. printf() and scanf() are examples for formatted input and output functions.

C. All of the above

D. None of the above

Answer for Self Assessment

1. B 2. B 3. C 4. D 5. B

6. D 7. C 8. D 9. B 10. B

11. A 12. C 13. A 14. D 15. B

Review Questions

1. Define stdin, stdout, and stderr.

2. Differentiate the followings:

(a) printf() and puts()

(b) getche() and getch()

(c) scanf() and gets()

3. How format string is associated with printf()? Discuss the various possible components of a format string in detail.

4. What happens if one uses variables in scanf() without using the address of operator (&)?

Discuss.

5. An amount of rupees, say R, is deposited in a bank for Y years, which pays simple interest at the rate of 'rt' annually. Write a C program that prints the amount after Y years

6. Write down two functions xgets() and xputs() which work similar to the standard library functions gets() and puts().

7. What is the differences between getchar(), fgetchar(), getch() and getche()? With the help of suitable example.

8. Write down two functions xgets() and xputs() which work similar to the standard library functions gets() and puts().

9. Write down a function getint(), which would receive a numeric string from the keyboard, convert it to an integer number and return the integer to the calling function. A sample usage of getint() is shown below:

```
main()
{
    int a ;
    a = getint( ) ;
    printf ( "you entered %d", a )
}
```

10. What is the differences between getchar(), fgetchar(), getch() and getche()?

**Further Readings**

Ashok N. Kamthane, "Programming with ANCI & Turbo C", Pearson Education, Year of Publication, 2008.

B.W. Kernighan and D.M. Ritchie, "The Programming Language", Prentice Hall of India, New Delhi.

Byron Gottfried, "Programming with C", Tata McGraw Hill Publishing Company Limited, New Delhi.

Greg W Scragg, Genesco Suny, Problem Solving with Computers, Jones and Bartlett, 1997.

R.G. Dromey, Englewood Cliffs, N.J., How to Solve it by Computer, Prentice-Hall International, 1982.

YashvantKanetkar, Let us C

**Web Links**

<https://www.tutorialspoint.com/index.htm>

www.webopedia.com

www.web-source.net